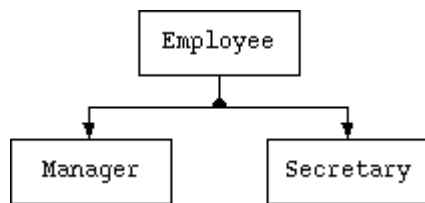


Konkret exempel

Vi skall nu se på ett enkelt konkret exempel. Vi kan tänka oss att vi vill på något sätt organisera de anställda på ett företag *Foo Factory* i olika klasser med olika uppgifter. Vi tänker oss att den viktiga information vi vill lagra om varje anställd är namn och lön. Förutom normala anställda vill vi även ha med chefer och sekreterare. Vi får då klasshierarkin:

Figur 15-1. Klasshierarki



Vi kan först definiera *basklassen* `Employee`:

```
class Employee {
public:
    // konstruktor
    Employee (string Name, float Salary);

    // returnera data
    string name ();
    float salary ();

private:
    // data om den anställda
    string m_Name;
    float m_Salary;
};

Employee::Employee (string Name, float Salary) {
    m_Name = Name;
    m_Salary = Salary;
}

string Employee::name () {
    return m_Name;
}

float Employee::salary () {
    return m_Salary;
}
```

Då man skapar ene `Employee` ger man med som parametrar till konstruktorn personens namn och lön. Vi har endast en konstruktor och inga möjligheter att ändra vare sig lön eller namn på en anställd. Vi skall nu göra vår första ärvning och skapa klassen `Manager` som ärver `Employee`. Vi antar för enkelhetens skull att varje `Manager` är chef för 10 anställda:

```
class Manager : public Employee {
public:
```

```
// konstruktor
Manager (string Name, float Salary);

private:
    // de anställda chefen ansvarar för
    Employee m_Employees[10];
};
```

Man definierar alltså ärvning genom att skriva den ärvda klassens namn efter den nya klassens namn. Man sätter ett `:` som separator. Nyckelordet `public` förklaras närmare i [avsnittet Typer av ärvning](#). I övrigt så ser klassdefinitionen helt normal ut. Nu kommer dock vårt första problem då vi ska skriva koden för konstruktorn för `Manager`. Vi måste kunna förmedla parametrar till basklassen `Employee` så vi kan ge värden åt `m_Name` och `m_Salary`. Det finns ett enkelt sätt att skicka med parametrar till en superklass:

```
Manager::Manager (string Name, float Salary) : Employee (Name, Salary) {
    // initialisera vektor med anställda
    ...
}
```

Man kan alltså skicka med parametrar till en superklass genom att placera ett `:` och klassnamnet efter konstruktorimplementationen. Varför gör man det och när måste man det?

Subklasser och konstruktorer

Då man skapar en subklass skapar man även implicit en basklass. Eftersom en `Manager` är även en `Employee` måste vi se till att vår klass `Manager` även initialiseras att vara en `Employee`. I C++ anropas alltid konstruktorer för basklasser då en subklass skapas. Ordningsföljden är enkel. Först anropas superklassers konstruktorer i ordningsföljd från den första och vidare mot basklassen. I vårt fall anropas konstruktorn för `Employee` först med de parametrar som gavs med, därefter konstruktorn för `Manager`. Om basklassen inte behöver några parametrar kan man lämna bort `:` och klassnamnet (i vårt fall `: Employee (Name, Salary)`) från implementationen av konstruktorn. Om basklassen däremot kräver parametrar måste man ha en dylik konstruktion. Man måste alltså alltid passa in konstruktorn så att den passar in med basklassens konstruktor. Basklassens konstruktor körs dock alltid, även om den behöver parametrar eller ej.

Man kan alltså anta och bygga på att konstruktorn för `Employee` har körts då den egentliga koden för konstruktorn för `Manager` körs.

Vi kan nu definiera den tredje klassen arbetare i vårt företag, nämligen `Secretary`. En dylik är även en `Employee`, men har som extra information den `Manager` som han/hon är sekreterare åt. Vi kan säga att klassen `Secretary` *has-a* `Manager`. Vi kan definiera klassen på följande sätt:

```
class Secretary : public Employee {
    // konstruktor
    Secretary (Manager Task, string Name, float Salary);

private:
    // sekreterares 'arbetsuppgift', d.v.s chef
    Manager m_Task;
};

Secretary::Secretary (Manager Task, string Name, float Salary) : Employee (Name,
    m_Task = Task;
}
```

Subklasser och destruktorer

Då objekt förstörs anropas deras destruktorer. I en klasshierarki fungerar det på samma sätt. Jämfört med konstruktorer anropas destruktorer i omvänd ordning, d.v.s. subklassernas destruktorer först, sedan basklassernas. I vårt fall då vi förstör en `Secretary` anropas först destruktorn för `Secretary`, därefter destruktorn för `Employee` och till sist destruktorn för den hypotetiska klassen `Person`. Destruktorer kan alltså även anta att allt som basklassend efinierat finns intakt, medan en basklass vet att en subclass redan är förstörd då subclassens destruktör anropas.

Anropa subklassers metoder

Vi kan nu skapa en metod som skriver ut lite information om en `Manager`. Informationen skall inkludera namn, lön och namnet på alla `Employee`:s som personen är chef för. Vi behöver då en ny metod till klassen `Manager`, låt oss kalla den `print()`.

```
class Manager : public Employee {
    ...
    // ny metod för att skriva ut information
    void print ();
    ...
}

Manager::print () {
    // skriv ut namn och lön
    cout << "Namn: " << name () << ", lön: " << salary () << endl;

    // skriv ut alla 10 anställda
    cout << "Chef för: " << endl;
    for ( int Index = 0; Index < 10; Index++ ) {
        cout << " m_Employees[Index].name () << endl;
    }
}
```

Vi ser nu att det verkar gå fint för en subclass att anropa metoder i superklassen. Det är exakt så som det är menat, subclasser skall använda sig av den funktionalitet som finns i superklasser. I detta fall är det endast frågan om metoderna `salary()` och `name()` från klassen `Employee`. Eftersom en `Manager` även är en `Employee` kan man för ett sådant objekt även anropa dessa metoder. Om man antar att klassen `Employee` vore ärvd från någon tredje klass, t.ex. `Person`, och där skulle finnas metoden `age()`, kunde även denna metod anropas. Subklasser kan alltid anropa superklassers metoder, om man inte skyddat dem speciellt för att förhindra detta (se [avsnittet Typer av ärvning](#)).

Diskussion

Denna klasshierarki är ganska långt värdelös, eftersom klasserna inte innehåller några egentliga metoder för att verkligen göra något, eller en viktig data. Den existerar dock endast för att illustrera hur man kan ärva klasser från varandra.

[Förutgående](#)
Ärvning

[Hem](#)
[Upp](#)

[Nästa](#)
Överlagring av metoder